NeuroMIMO

Problem, Solution and Working

The whole project, explained from start to finish

AI-enhanced channel estimation for 2×2 MIMO-OFDM wireless systems: what problem it solves, how the solution is designed, and how every part works.

October 30, 2026

Contents

The Project in One Page 2

1. The Problem 3

2. The Solution 5

3. How It Works 7

4. What the Solution Achieved 10

5. Conclusion 11

The Project in One Page

NeuroMIMO is a software platform that simulates a modern wireless link (the kind used in 5G and Wi-Fi) and studies how the receiver can work out what the air has done to the signal. It compares two classical methods with an artificial-intelligence method and shows everything in an interactive web dashboard.

THE PROBLEM

A radio signal reaches the receiver through many reflected paths and picks up noise, so every frequency is weakened and rotated differently. To recover the data, the receiver must estimate this **channel** from a few known **pilot** symbols, fast and accurately. The simple method (Least Squares) is noisy; the best classical method (LMMSE) needs knowledge of the channel's statistics that a real receiver does not have.

THE SOLUTION

Build a complete 2×2 MIMO-OFDM simulator and compare three channel estimators on exactly the same channels: **Least Squares**, **Linear MMSE**, and a **residual deep neural network** that takes the noisy LS estimate and learns to correct it. Measure accuracy, bit errors with confidence intervals, and speed, and serve it all through a FastAPI backend and a React dashboard.

THE RESULT

At 0 dB SNR (very noisy) LMMSE is best (−9.05 dB), and the DNN (−8.67 dB) beats LS (−8.38 dB) *without* needing channel statistics. At high SNR the DNN levels off near −25 dB while LS and LMMSE reach −38.64 dB. Every estimator runs in under 1 ms per frame. The project shows where learning helps, where it does not, and why.

1. The Problem

1.1 Background: how modern wireless links work

Phones, Wi-Fi routers and 5G base stations all use two key technologies. **MIMO** (Multiple-Input Multiple-Output) uses several antennas at each end to send several data streams at the same time on the same frequency, multiplying the data rate. **OFDM** (Orthogonal Frequency-Division Multiplexing) splits the frequency band into many narrow *subcarriers* that each carry part of the data. Together, MIMO-OFDM is the physical layer of 4G LTE, 5G NR and Wi-Fi 6/7.

1.2 What goes wrong in the air

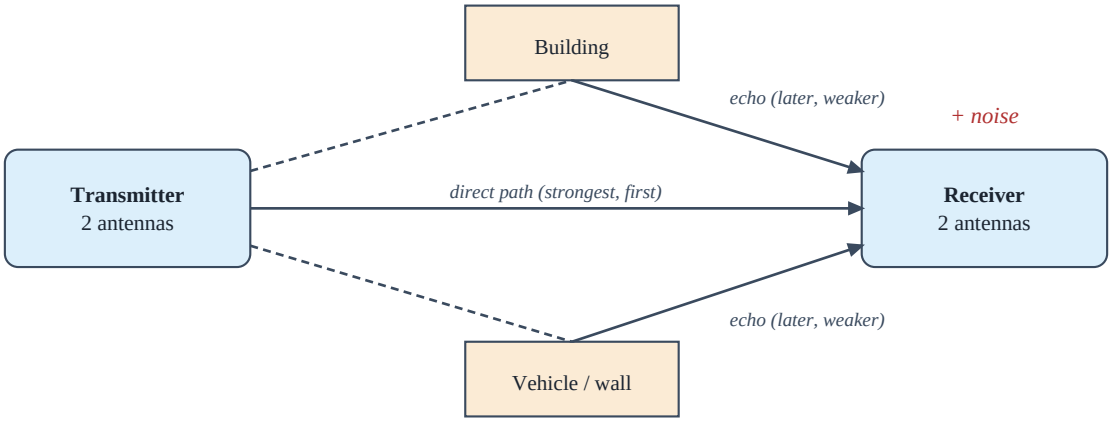


Figure 1: The signal arrives through a direct path and delayed echoes, plus noise.

- **Multipath:** the signal bounces off buildings, walls and vehicles, so several copies arrive at slightly different times. They add up constructively at some frequencies and cancel at others (*frequency-selective fading*).
- **Fading:** with no clear line of sight the combined signal strength varies randomly (*Rayleigh fading*).
- **Noise:** the receiver's electronics add random noise. The weaker the signal compared with the noise (low SNR), the harder it is to read.
- **MIMO mixing:** each receive antenna hears *both* transmit antennas at once, mixed together.

1.3 Why the receiver must estimate the channel

On each subcarrier k, what the receiver gets is the transmitted symbols multiplied by the channel, plus noise:

$$\mathbf{y}[k] = \mathbf{H}[k] \cdot \mathbf{x}[k] + \mathbf{n}[k]$$

Here $\mathbf{H}[k]$ is a 2×2 matrix of complex numbers: how each transmit antenna reaches each receive antenna on that frequency. To recover the data $\mathbf{x}$, the receiver has to undo $\mathbf{H}$, so it must first **estimate** it. A wrong estimate means wrongly decoded bits. The transmitter helps by sending **pilots**: known symbols on known subcarriers. Comparing what arrived with what was sent reveals the channel at those positions.

1.4 Why this is hard

Difficulty	In this project
Few measurements	Only 4 pilots per transmit antenna, but the channel is needed on all 64 subcarriers.
Many unknowns	Four separate channels (2 transmit × 2 receive antennas) per subcarrier.
Noise	At 0 dB SNR the noise is as strong as the signal, so pilot measurements are very noisy.
Unknown statistics	A real receiver does not know the channel's correlation or the exact noise level.
Real-time deadline	A 5G slot lasts about 1 ms, so the estimate must be computed in well under that.

1.5 Limits of the existing solutions

Method	Strength	Weakness
Least Squares (LS)	Simple, fast, needs no prior knowledge	Passes all the pilot noise into the estimate; poor at low SNR
Linear MMSE (LMMSE)	Best linear estimator; removes noise using statistics	Needs the channel correlation and noise level; matrix inverse is costly
Published deep learning	Can learn the channel's structure from data	Often shown only in favourable conditions, without confidence intervals or speed

Problem statement. How accurately, how reliably and how fast can a lightweight deep neural network estimate the channel of a 2×2 MIMO-OFDM link from pilot symbols, compared with LS and LMMSE, across noisy and clean conditions (0–30 dB SNR), and does it work without knowing the channel statistics?

2. The Solution

2.1 The idea

Instead of claiming that AI is better, NeuroMIMO builds a fair test bench. It simulates the whole radio link, runs every estimator on **exactly the same** channels and noise, and measures them with the same yardsticks. The AI estimator is designed to need only what a real receiver has: the noisy pilot measurements.

2.2 What was built

Component	What it is	Why it is needed
MIMO-OFDM simulator	Transmitter, channel and receiver in Python/NumPy	Creates realistic signals where the true channel is known, so errors can be measured exactly
Least Squares estimator	Received pilot ÷ known pilot, then DFT interpolation	The simple baseline every receiver can run
LMMSE estimator	Wiener filter using channel statistics and noise level	The classical best-case reference
Residual DNN estimator	PyTorch network that corrects the LS estimate	The learned method under test
Perfect-CSI reference	Uses the true channel	Shows the best possible bit error rate
Evaluation framework	Five experiments, confidence intervals, timing	Makes the comparison fair, honest and repeatable
Web platform	FastAPI backend + React dashboard	Lets anyone run, train and explore the results
DeepMIMO loader	Reads ray-traced channels from realistic scenes	Tests the methods beyond the built-in channel model

2.3 The key idea: a residual network

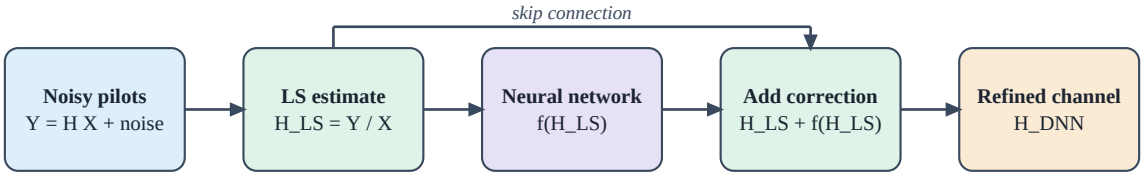


Figure 2: The DNN learns only the correction to LS.

The LS estimate is already roughly right; its problem is noise. So the network is not asked to find the channel from scratch. It takes the LS estimate as input and learns a **correction** that is added back:

$$\hat{H}_{DNN} = \hat{H}_{LS} + f_{\theta}(\hat{H}_{LS})$$

This is easier to learn, trains faster, and means the network starts from LS quality rather than from nothing. The network is small (330,112 parameters), so it runs in real time on an ordinary CPU.

2.4 Design decisions

Decision	Reason
2×2 antennas, 64 subcarriers	The smallest real MIMO case: fast to simulate and train on a laptop.
Comb pilots, 4 per antenna	Pilots in every symbol; 4 pilots are exactly enough for a 4-tap channel.
Orthogonal pilot positions	The two antennas use different subcarriers, so their pilots never collide.
16-sample cyclic prefix	Longer than the channel, so echoes do not spill into the next symbol.
Identical channels for all methods	Differences then come from the estimators, not from luck.
Train/validation/test split	The network is tested on channels it never saw during training.
Confidence intervals on BER	A finite test has uncertainty; the intervals show how much.

3. How It Works

This section follows one frame of data from the transmitter, through the air, to the receiver, then explains how the network learns and how results are measured.

3.1 The complete signal chain

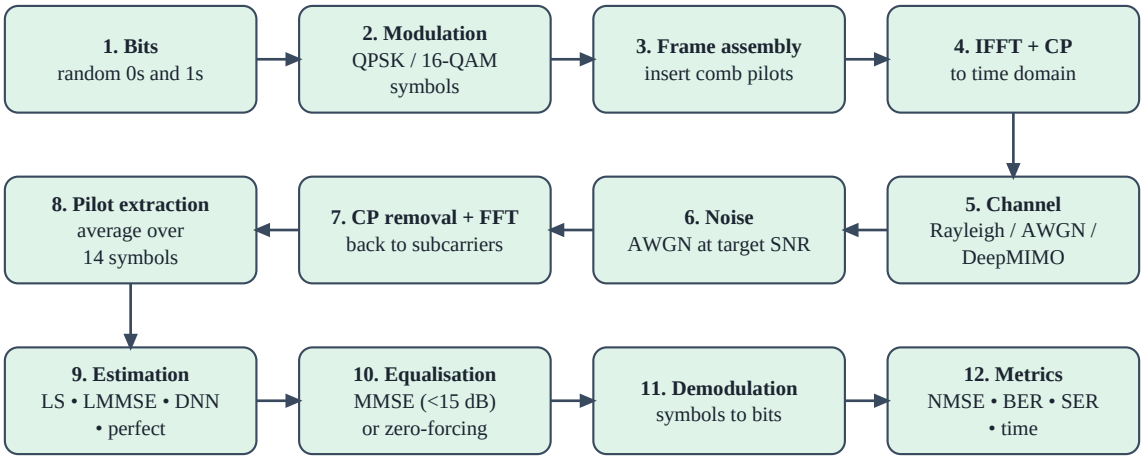


Figure 3: One frame from transmitter to metrics.

3.2 Transmitter (steps 1–4)

- **Bits:** random data, 3,136 bits per frame with QPSK (2 antennas × 14 symbols × 56 data subcarriers × 2 bits).
- **Modulation:** each pair of bits becomes one QPSK symbol (a point on a circle). Gray coding means a small error flips only one bit.
- **Frame assembly:** antenna 0 puts pilots on subcarriers 0, 16, 32, 48 and antenna 1 on 8, 24, 40, 56; each stays silent on the other's pilots. The other 56 subcarriers carry data.
- **IFFT + cyclic prefix:** the IFFT turns the 64 subcarriers into a time signal, and the last 16 samples are copied to the front as a guard against echoes.

3.3 The channel (steps 5–6)

- **Rayleigh multipath:** each of the 4 antenna pairs gets 4 random echo taps, the first strongest and later ones weaker (exponential power delay profile). The channel stays constant for the whole 14-symbol frame (block fading).
- **Noise:** Gaussian noise is added so that the received signal has exactly the requested SNR, from 0 dB (noise as strong as signal) to 30 dB (signal 1000 times stronger).
- **Optional realism:** instead of the random model, channels can come from DeepMIMO ray-tracing of real-looking scenes.

3.4 Receiver (steps 7–12)

- **CP removal + FFT:** the receiver drops the guard samples and returns to the 64 subcarriers.
- **Pilot extraction:** it reads the pilot subcarriers and averages each pilot over the 14 symbols, which cuts the noise 14 times (the channel does not change within the frame).
- **Channel estimation:** one of the estimators (Section 3.5) produces $\hat{H}$ for all 64 subcarriers and all four antenna pairs.

- **Equalisation:** the receiver undoes $\hat{H}$ on the data subcarriers to separate the two streams (MMSE equaliser below 15 dB, zero-forcing above).
- **Demodulation and metrics:** symbols become bits again; the receiver counts errors and compares $\hat{H}$ with the true channel.

3.5 The three estimators

Estimator	How it works	Needs
Least Squares	Divide each received pilot by the known pilot ($\hat{H} = Y/X$), then fill in the other subcarriers with DFT interpolation.	Nothing extra
Linear MMSE	Smooth the LS pilot estimates with a Wiener filter built from the channel's frequency correlation and the pilot noise level ($\sigma^2/14$ after averaging).	Channel statistics + noise level
Residual DNN	Feed the LS estimate (512 numbers) through a $512 \to 256 \to 128 \to 256 \to 512$ network and add its output as a correction.	A trained model

3.6 How the network learns

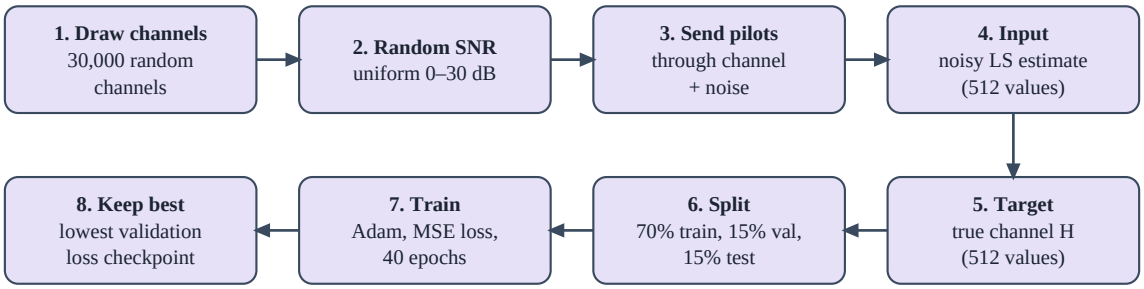


Figure 4: Training the residual DNN.

Because this is a simulation, the true channel is known for every example, so the network can be taught the right answer. It only ever sees what a real receiver would have (the noisy LS estimate), and it is tested on channels it never saw during training.

3.7 How the results are measured

- **NMSE:** how far $\hat{H}$ is from the true channel, relative to the channel's power, in dB. −10 dB means 10% error, −20 dB means 1%.
- **BER / SER:** the fraction of wrong bits and symbols after the whole receiver.
- **95% confidence intervals:** Wilson intervals on every BER point; when no errors are seen, the “rule of three” gives an upper bound instead of claiming zero.
- **Fairness:** the random generator is reset with the same seed for every estimator, so all of them face identical channels and noise.
- **Speed:** each estimator is timed 100 times after 10 warm-up runs.

3.8 How you use it

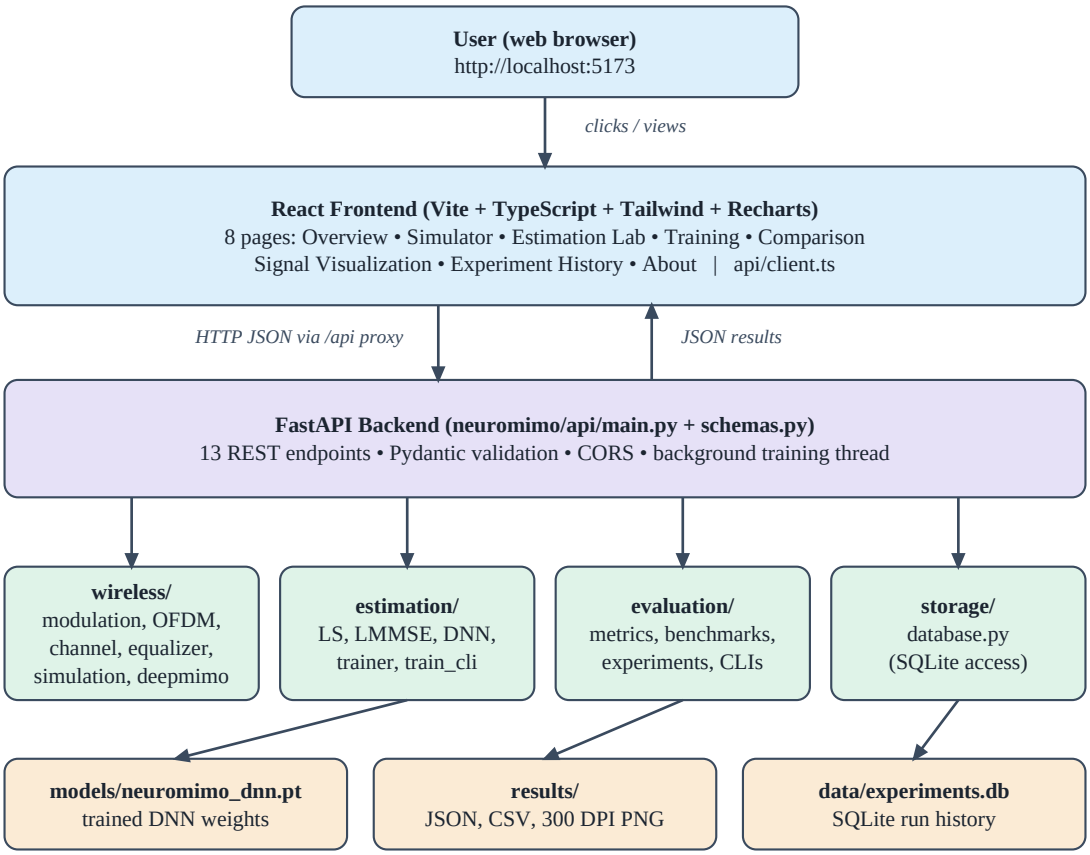


Figure 5: The web platform around the simulator.

You open the dashboard in a browser, choose settings and press a button. The React page sends the settings to the FastAPI backend (for example `POST /api/simulation/run`), the backend runs the signal chain of Figure 3, saves the run in SQLite and returns the numbers, and the page draws the charts. Training runs in the background while the page shows the loss curve live.

4. What the Solution Achieved

SNR	LS	LMMSE	DNN	Best
0 dB	−8.38 dB	−9.05 dB	−8.67 dB	LMMSE
10 dB	−18.24 dB	−18.44 dB	−18.36 dB	LMMSE
20 dB	−28.95 dB	−28.93 dB	−25.31 dB	LS
30 dB	−38.64 dB	−38.64 dB	−25.96 dB	LS

Table: channel estimation error (NMSE, lower is better), Rayleigh channel, QPSK.

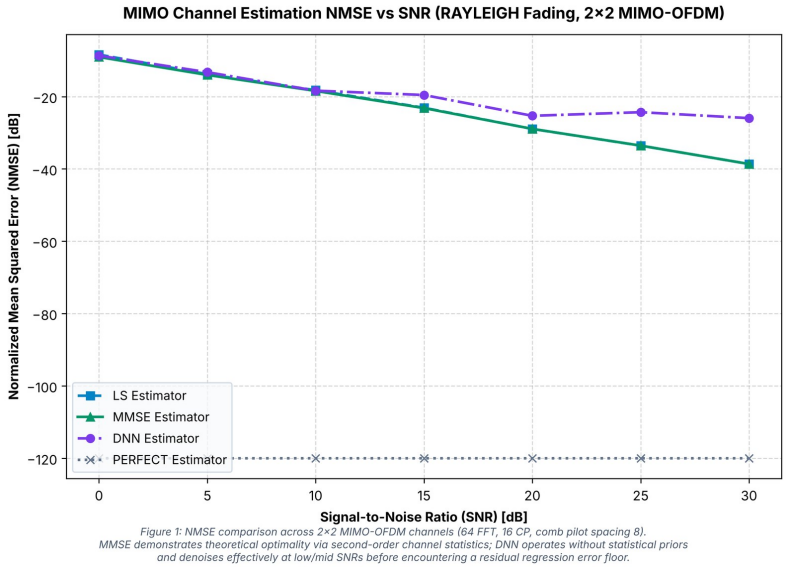


Figure 6: NMSE versus SNR for every estimator.

Answering the problem statement

- **Accuracy in noisy conditions:** LMMSE is best (−9.05 dB at 0 dB), as theory predicts. The DNN (−8.67 dB) beats LS (−8.38 dB) and comes close to LMMSE **without knowing the channel statistics**, which a real receiver lacks.
- **Accuracy in clean conditions:** the DNN levels off near −25 dB while LS and LMMSE keep improving to −38.64 dB. One small network trained across the whole 0–30 dB range cannot match LS when the noise is tiny.
- **Speed:** LS 0.12 ms, LMMSE 0.40 ms, DNN 0.65 ms per frame on a CPU, all well inside a 1 ms 5G slot.
- **Reliability:** every result is reproducible from fixed seeds and covered by 88 automated tests.

Problems found and fixed during the project

- **LMMSE noise level:** LMMSE was first given the noise level before pilot averaging (σ^2 instead of $\sigma^2/14$), which made it look worse than LS. Fixing it made LMMSE the best estimator at low SNR, matching theory.
- **Overwritten model:** a test had overwritten the trained model with a 2-epoch test model. The test was fixed, and the DNN was retrained on 30,000 channels.

5. Conclusion

> NeuroMIMO shows that a small residual neural network can estimate a MIMO-OFDM channel better than Least Squares in noisy conditions and close to LMMSE, **without the channel statistics LMMSE needs**, and fast enough for real time. It also shows honestly where it falls short: at high SNR it has an error floor. The platform makes this comparison fair, repeatable and easy to explore.

Limitations

- Simulated channels by default (the DeepMIMO loader is the step toward realism).
- Small 2×2 system, block fading with no Doppler, perfect synchronisation.
- The DNN's high-SNR error floor.

Future work

- Give the SNR to the network as an input, or train one model per SNR range, to remove the floor.
- Use a convolutional network across time and frequency.
- Add moving users (Doppler, Jakes model) and larger antenna arrays (4×4, 8×8).
- Evaluate on DeepMIMO ray-traced channels and learn the pilot placement.